

Konstruksi Basis Pengetahuan Modular pada OMNI-RAG Multi-Domain: Evaluasi Chunker, Indexer, dan Legal Knowledge Graph

Moh Fairuz Alauddin Yahya
Program Studi Informatika, Institut Teknologi Bandung
Bandung, Indonesia
13522057

Abstract—Pipeline *Retrieval-Augmented Generation* (RAG) harus mengubah dokumen yang beragam menjadi unit informasi yang dapat dicari sebelum model bahasa menyusun jawaban. Paper ini menyajikan rancangan dan evaluasi kontribusi individual untuk konstruksi basis pengetahuan pada OMNI-RAG, yaitu arsitektur multi-domain berbasis *plugin*. Fokus penelitian adalah tiga proyeksi dari dokumen kanonik yang sama: *chunker*, *indexer*, dan *knowledge graph*. Kontrak *CanonicalDocument*, *CanonicalChunks*, *CanonicalIndex*, dan *CanonicalGraph* memisahkan orkestrator dari strategi pemrosesan, sedangkan *profile*, manifest, dan provenance menjaga pemilihan komponen serta keterlacakan hasil. Eksperimen *one factor at a time* menggunakan sembilan manual Netgear (1.400 halaman, 200 pertanyaan bilingual) dan korpus hukum (256 dokumen, 25 pertanyaan terverifikasi). Pada $K = 30$, *structure-aware* menghasilkan *recall* 0,740 pada manual, sedangkan *legal-structure dense* menghasilkan 0,6854 pada korpus hukum. Dense indexer BAAI/BGE-M3 memperoleh *recall* 0,785 pada manual dan 0,6854 pada hukum; hybrid memiliki MRR hukum tertinggi, 0,5439. Traversal graf hingga kedalaman dua mencapai *neighborhood recall* 1,0000 pada konfigurasi *sliding*. Hasil menunjukkan bahwa pemeliharaan struktur dan pencocokan semantik meningkatkan cakupan konteks, tetapi konfigurasi terbaik bergantung pada domain serta metrik yang diprioritaskan.

Index Terms—OMNI-RAG, basis pengetahuan, chunking, indexing, knowledge graph, plugin, retrieval augmented generation.

I. PENDAHULUAN

Sistem tanya jawab berbasis *large language model* (LLM) membutuhkan sumber eksternal agar jawaban tidak hanya bergantung pada pengetahuan parametrik model. *Retrieval-Augmented Generation* (RAG) menyediakan sumber tersebut dengan mengambil potongan dokumen sebelum proses generasi [1, 2]. Tantangannya bukan sekadar menyimpan dokumen. Dokumen hukum memiliki hierarki Bab, Bagian, Pasal, Ayat, Huruf, dan Angka, sedangkan *User Manual* berisi struktur yang lebih bebas seperti heading, langkah prosedural, tabel, dan peringatan. Jika batas potongan, representasi indeks, atau hubungan antardokumen dibentuk tanpa memperhatikan struktur, konteks yang dibutuhkan dapat terpisah atau sulit ditemukan.

Paper ini mengambil satu kontribusi teknis dari platform OMNI-RAG yang dikembangkan dalam proyek Capstone empat orang. Konteks tersebut hanya diperlukan untuk menempatkan jalur konstruksi basis pengetahuan di antara komponen sistem lain; pembahasan dan evaluasi di sini dibatasi pada pekerjaan individu penulis. Jalur yang dipelajari adalah pemrosesan *offline* dari dokumen kanonik menjadi potongan, indeks, dan graf yang dapat dipakai oleh tahap *retrieval*.

Tiga masalah penelitian dirumuskan sebagai berikut.

- 1) Bagaimana strategi *chunker* dapat membentuk unit informasi yang mempertahankan konteks struktural pada dokumen dengan karakteristik berbeda?
- 2) Bagaimana strategi *indexer* sparse, dense, dan hybrid memengaruhi kelengkapan serta urutan konteks yang ditemukan?
- 3) Bagaimana *knowledge graph* dibangun dari unit yang sama dan digunakan untuk memperluas jangkauan dokumen tanpa mengaburkan provenance sumber?

Kontribusi paper ini adalah (1) rancangan kontrak artefak kanonik yang menghubungkan parser, *chunker*, *indexer*, dan *knowledge graph*; (2) implementasi arsitektur *plugin* dan *profile* yang memungkinkan strategi diganti tanpa mengubah orkestrator; (3) legal *knowledge graph* deterministik yang merepresentasikan hubungan perubahan peraturan; dan (4) evaluasi terkontrol pada korpus manual produk dan peraturan pendidikan Indonesia. Evaluasi tidak menggabungkan hasil lintas konfigurasi: setiap konfigurasi memakai korpus, pertanyaan, parser, dan model embedding yang sama ketika satu faktor sedang dibandingkan.

Ruang lingkup paper adalah konstruksi basis pengetahuan dan dampaknya pada *retrieval* serta generation. Tahap akuisisi dan parsing diperlakukan sebagai pemasok dokumen kanonik, sedangkan komponen percakapan berada di luar klaim penelitian ini. Dengan batas tersebut, bagian berikutnya membahas landasan teknis, rancangan pipeline, metode eksperimen, hasil, dan implikasinya.

II. LANDASAN DAN POSISI PENELITIAN

A. Unit Dokumen dan Chunking

Chunking menentukan unit yang akan disimpan, dicari, dan diberikan kepada generator. Potongan terlalu besar membawa noise dan melampaui anggaran konteks, sedangkan potongan terlalu kecil dapat memisahkan definisi, syarat, pengecualian, atau hubungan antarbagian. Studi NitiBench menunjukkan bahwa pemotongan berbasis struktur dapat memberikan hasil berbeda dari strategi generik pada pertanyaan hukum [3]. Karena itu, batas potongan diperlakukan sebagai keputusan desain yang perlu diuji pada domain target, bukan sebagai parameter universal.

Penelitian ini membandingkan *sliding-window*, *recursive*, *generic structure-aware*, dan *legal structure-aware*. Strategi generik memanfaatkan outline parser dan dapat membawa konteks heading induk. Strategi legal memusatkan unit pada Pasal, mempertahankan konteks Bab atau Bagian, lalu dapat membentuk *child chunk* sebagai kandidat dan *parent chunk* sebagai konteks lengkap.

B. Indexer dan Representasi Pencarian

Indexer leksikal seperti BM25 memberi bobot pada istilah yang jarang dan menormalisasi panjang dokumen [4]. Dense retrieval memetakan kueri dan unit teks menjadi vektor sehingga parafrasa dapat ditemukan meskipun tidak memiliki kata yang sama [5]. BAAI/BGE-M3 dipakai sebagai model embedding multibahasa untuk eksperimen ini [6]. Hybrid retrieval menggabungkan daftar leksikal dan dense melalui Reciprocal Rank Fusion [7].

Perbedaan ketiga representasi penting pada dokumen hukum. Nomor peraturan dan nomor Pasal membutuhkan kecocokan literal, sementara pertanyaan pengguna dapat memparafrasekan norma. Skor dense dipahami sebagai kedekatan representasi, bukan putusan bahwa dua teks memiliki makna hukum yang identik. Identitas dokumen, struktur, metadata, dan provenance tetap menjadi dasar yang dapat diperiksa.

C. Knowledge Graph sebagai Proyeksi Pelengkap

knowledge graph merepresentasikan entitas dan hubungan sebagai graf $G = (V, E)$ dengan sisi bertipe. Graf dapat menampung hubungan eksplisit yang tidak selalu muncul sebagai kemiripan teks, misalnya peraturan yang mengubah, menetapkan, atau mencabut peraturan lain. Paper ini membedakan dukungan graf opsional dari *GraphRAG*: indeks teks tetap menghasilkan seed, lalu graf memperluas lingkungan dokumen seed; graf bukan pengganti pencarian teks atau mekanisme ringkasan komunitas utama [8, 9].

Desain modular diperlukan karena strategi yang baik pada satu domain belum tentu baik pada domain lain. Modular RAG memisahkan komponen yang dapat diganti, sedangkan kontrak antarmuka menjaga agar perubahan strategi tidak menyebar ke orkestrator [10, 11]. Penelitian ini menerapkan prinsip tersebut pada artefak pengetahuan dan mengujinya melalui hasil retrieval yang terukur.

TABLE I

KONTRAK ARTEFAK PADA PIPELINE KONSTRUKSI BASIS PENGETAHUAN

Artefak	Isi dan konsumen
CanonicalDocument	Identitas dokumen, blok hierarkis, teks, lokasi sumber, metadata, dan provenance; dibaca <i>chunker</i> .
CanonicalChunks	Identitas chunk, tipe retrieval/context, teks, parent, posisi, fingerprint, relasi, dan provenance; dibaca <i>indexer</i> dan <i>knowledge graph</i> .
CanonicalIndex	Identitas chunk, teks, metadata, keluaran lexical atau embedding, model, dan versi; dimaterialisasi pada OpenSearch.
CanonicalGraph	Simpul, sisi bertipe, bukti, cakupan dokumen, dan provenance; dimaterialisasi pada Neo4j.

III. RANCANGAN PIPELINE KONSTRUKSI BASIS PENGETAHUAN

A. Batas Sistem dan Artefak Kanonik

Jalur yang dibangun menerima keluaran parser berupa dokumen kanonik, lalu menjalankan *chunker* dan memproyeksikan hasilnya secara paralel ke *indexer* serta *knowledge graph*. Pemisahan ini mengikuti prinsip modular RAG: worker membaca referensi artefak dari object storage, sedangkan pesan hanya membawa identitas run, node, dan kunci idempotensi. Dengan demikian, kegagalan atau pengulangan satu tahap tidak mengharuskan seluruh dokumen diproses ulang.

Empat kontrak menjadi batas integrasi.

profile mengikat datasource, urutan node, implementasi *plugin*, konfigurasi, dan versi kontrak. Plugin Registry menyimpan identitas, tipe komponen, versi, skema konfigurasi, dan entry point. Resolver memuat implementasi berdasarkan manifest lalu memvalidasi kompatibilitas sebelum worker menjalankan proses. Penggantian strategi cukup mengubah node *profile* atau menambahkan *plugin* baru yang memenuhi kontrak; orkestrator tetap membaca dan menulis jenis artefak yang sama.

Setiap eksekusi menyimpan pipeline run dan node attempt, termasuk producer, konfigurasi, jumlah masukan, jumlah keluaran, status, durasi, kesalahan, dan referensi artefak. Metadata ini bukan hasil retrieval, tetapi bagian dari mekanisme reproduksi: dua konfigurasi dapat dibandingkan tanpa kehilangan identitas sumber, model, atau versi *plugin*.

B. Strategi Chunker

Sliding-window menggunakan jendela 512 karakter dan overlap 51 karakter. *Recursive* memecah paragraf, kalimat, kata, lalu karakter ketika batas ukuran terlampaui. *Generic structure-aware* memanfaatkan outline parser, memilih target `text_block`, dan membawa heading induk sebagai konteks. *Legal structure-aware* mengikuti hierarki Bab, Bagian, Pasal, Ayat, Huruf, dan Angka. Satu Pasal dapat menjadi parent chunk yang utuh, sedangkan child chunk dipakai sebagai kandidat pencarian ketika isi Pasal terlalu panjang.

Parent hydration pada strategi legal memisahkan ketepatan pencarian dari kelengkapan konteks. Child membawa bagian yang cocok dengan kueri; ketika child ditemukan, resolver dapat memuat parent Pasal beserta metadata Bab atau Bagian. Identitas parent, child, dan source block tetap disimpan sehingga konteks tidak dibentuk ulang dari teks yang berbeda.

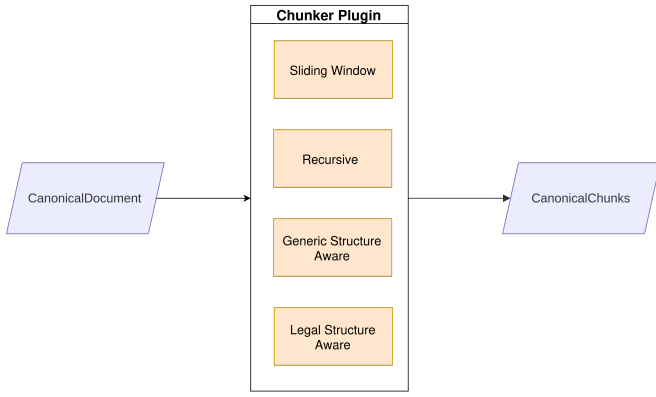


Fig. 1. Kontrak *chunker*: satu CanonicalDocument dapat diproses oleh beberapa strategi dan menghasilkan CanonicalChunks.

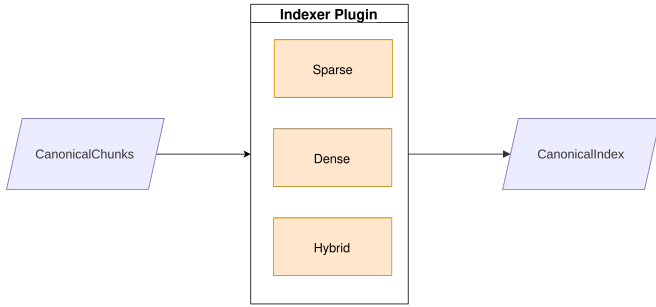


Fig. 2. Kontrak *indexer* dari CanonicalChunks ke CanonicalIndex.

C. Strategi Indexer

Indexer hanya memasukkan chunk bertipe *retrieval* atau *both* sebagai kandidat. Chunk bertipe *context* dipertahankan untuk hydration dan sitasi, tetapi tidak dicari mandiri. Sparse indexer menggunakan BM25-compatible lexical fields. Dense indexer membentuk embedding BAAI/BGE-M3 dan menyimpan vector field. Hybrid indexer mengambil dua daftar kandidat dan menggabungkannya dengan Reciprocal Rank Fusion tanpa mengubah artefak chunk.

OpenSearch menjadi penyimpanan proyeksi retrieval, sementara salinan CanonicalIndex tetap disimpan sebagai artefak. Upsert membawa chunk ID, document ID, profile ID, metadata, model, dan provenance. Jika materialisasi gagal, artefak dapat dikirim ulang tanpa menghitung embedding dari awal. Penghapusan profile atau dokumen juga menghapus proyeksi yang sesuai agar indeks tidak menyisakan konteks lama.

D. Legal Knowledge Graph

knowledge graph opsional menerima CanonicalChunks yang sama dengan indexer, bukan hasil pemotongan atau sumber terpisah. Implementasi eksperimen adalah graf legal deterministik. Tipe simpulnya adalah Regulation, RevisePasal, dan Pasal; relasi internal HAS menjaga struktur dokumen, sedangkan relasi antarperaturan mencakup AMENDS,

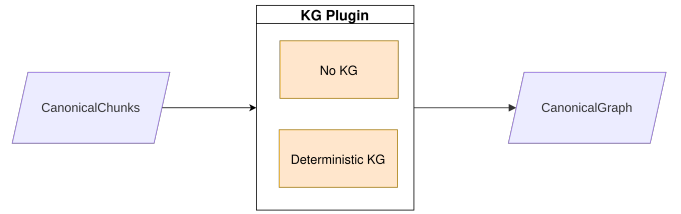


Fig. 3. Kontrak *knowledge graph* opsional dari CanonicalChunks ke CanonicalGraph.

TABLE II
KORPUS DAN UNIT EVALUASI

Korpus	Cakupan dan acuan
User Manual	9 manual Netgear, 1.400 halaman, 100 slot bilingual, 200 pertanyaan, satu context reference per slot.
Peraturan pendidikan	256 dokumen (49 memuat reference dan 207 noise), 25 pertanyaan terverifikasi, 148 baris reference, 144 context unik, 45 dokumen seed.

ESTABLISHES, dan REVOKES. Relasi hanya dibuat jika identitas dan status sumber eksplisit pada metadata.

Resolver legal tidak mencocokkan nomor Pasal saja. Untuk operasi perubahan, resolver memeriksa pasangan identitas peraturan target dan nomor Pasal; untuk penyisipan, resolver juga melihat nomor tetangga. Isi Pasal hasil resolusi dimuat dari artefak parser. Saat online, hasil top- K indexer menjadi seed, lalu traversal global dibatasi dua lompatan, arah relasi, tipe relasi, dan cakupan profile. Hasil traversal dipetakan kembali ke chunk ID dan dideduplikasi. Dengan mekanisme ini, graf memperluas lingkungan dokumen tanpa menggantikan bukti teks.

E. Klaim yang Diuji

Rancangan menghasilkan tiga hipotesis operasional. Pertama, pemeliharaan struktur meningkatkan cakupan context reference dibandingkan fixed-window. Kedua, dense retrieval meningkatkan recall ketika kueri memparafrasekan istilah sumber, sedangkan hybrid dapat menguntungkan posisi hit pertama. Ketiga, traversal legal graph memperluas jangkauan dokumen tetangga, tetapi neighborhood recall harus dibaca terpisah dari context-reference recall. Ketiganya diuji dengan konfigurasi, corpus, dan cutoff yang ditetapkan sebelum perbandingan.

IV. METODE EKSPERIMEN

A. Lingkungan dan Korpus

Pipeline dijalankan dengan Python 3.13 pada worker asinkron. PostgreSQL menyimpan status dan metadata eksekusi, object storage menyimpan artefak, RabbitMQ membawa handoff antartahap, OpenSearch 3.3.0 menyediakan lexical dan vector search, dan Neo4j 5.26 menyimpan graf. Embedding menggunakan BAAI/BGE-M3 pada CPU. Infrastruktur, parser, model, dan jumlah kandidat dibuat sama untuk setiap perbandingan komponen.

Pada corpus manual, sembilan dokumen meliputi CM2000, LM1200, R6350, R7000, RAX120, RAX50, RAXE500, RBK852, dan XR500. Pertanyaan dibentuk dari slot yang disampling terstratifikasi menurut dokumen dan posisi halaman, kemudian setiap slot diberi varian Inggris dan Indonesia. Pada corpus hukum, 49 dokumen yang memuat reference dipertahankan sebagai inti; 207 noise dipilih secara stratified berdasarkan kelompok bentuk, status, rentang tahun, dan panjang sehingga semua konfigurasi melihat datasource 256 dokumen yang sama.

B. Protokol Perbandingan

Eksperimen memakai *one factor at a time* (OFAT). Pada kelompok chunker, datasource, parser, embedding, dan indexer dikunci; hanya strategi segmentasi yang berubah. Pada kelompok indexer, chunker dan seluruh input dikunci; hanya representasi lexical, dense, atau hybrid yang berubah. Jalur direct retrieval mematikan reranker dan *knowledge graph* agar metrik retrieval tidak tercampur dengan tahap hilir. Sweep cutoff menggunakan $K = 5, 10, \dots, 60$; $K = 30$ dipilih sebagai cutoff kandidat bersama untuk membaca trade-off coverage dan beban downstream. Generation menggunakan sepuluh evidence teratas ($K = 10$).

Pertanyaan manual dihitung gabungan serta terpisah menurut bahasa. Pertanyaan hukum mencakup pendirian dan status institusi, kewenangan, akses mahasiswa, kualifikasi staf, penjaminan mutu, serta kepatuhan. Semua hasil dicatat per konfigurasi; hasil kandidat dari konfigurasi berbeda tidak digabungkan.

C. Metrik

Misalkan G_q adalah himpunan context reference acuan untuk pertanyaan q , sedangkan $R_q(K)$ adalah himpunan chunk unik yang dikembalikan sampai cutoff K . Recall dan precision dihitung sebagai

$$\begin{aligned} \text{Recall @ } K &= \frac{1}{|Q|} \sum_{q \in Q} \frac{|G_q \cap R_q(K)|}{|G_q|}, \\ \text{Precision @ } K &= \frac{1}{|Q|} \sum_{q \in Q} \frac{|G_q \cap R_q(K)|}{K}. \end{aligned} \quad (1)$$

MRR menilai posisi reference pertama. Jika r_q adalah posisi tersebut, kontribusinya $1/r_q$ dan bernilai nol ketika reference tidak ditemukan. NDCG menggunakan label relevansi bertingkat 0, 1, dan 2 untuk membedakan konteks yang tidak mendukung, mendukung sebagian, dan mendukung langsung. Pada generation, *correctness* mengukur kelengkapan jawaban terhadap acuan, sedangkan *faithfulness* memeriksa dukungan klaim oleh evidence.

Untuk graf, reference dipetakan ke dokumen sumber. N_q adalah himpunan dokumen tetangga acuan dan $H_q(K)$ adalah hasil setelah seed retrieval dan traversal global hingga kedalaman dua. Neighborhood recall didefinisikan sebagai

$$\text{KGRecall @ } K = \frac{1}{|Q|} \sum_{q \in Q} \frac{|N_q \cap H_q(K)|}{|N_q|}. \quad (2)$$

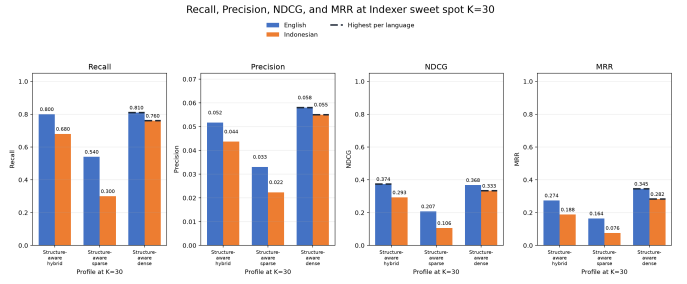


Fig. 4. Metrik indexer User Manual pada $K = 30$.

Metrik ini mengukur jangkauan lingkungan, bukan keberhasilan menemukan setiap context reference secara langsung. Oleh sebab itu, nilai graf dilaporkan terpisah dari Recall@K retrieval.

D. Konfigurasi yang Dibandingkan

Konfigurasi manual menggunakan satu context reference per slot sehingga perubahan Recall@K dapat dibaca langsung sebagai perubahan cakupan unit acuan. Konfigurasi hukum menggunakan 144 context reference unik; hubungan legal graph dibangun dari 17 relasi berarah dan 24 relasi tak berarah yang tersedia pada benchmark. Pemilihan $K = 30$ tidak menyatakan bahwa seluruh kurva telah datar, melainkan menyediakan titik kerja yang sama untuk membandingkan komponen dan meneruskan kandidat ke tahap berikutnya.

V. HASIL DAN PEMBAHASAN

A. Hasil pada User Manual

Pada kelompok chunker, structure-aware hybrid menghasilkan Recall@30 sebesar 0,740, lebih tinggi 0,055 daripada sliding-window dan 0,115 daripada recursive (Tabel IV). Keunggulan tersebut juga muncul pada precision dan NDCG. Sliding-window memiliki MRR sedikit lebih tinggi, 0,232 berbanding 0,231, sehingga pemeliharaan struktur meningkatkan jumlah reference yang ditemukan tanpa selalu menempatkan reference pertama pada posisi paling awal. Ketika cutoff dinaikkan ke 60, recall hanya berubah menjadi 0,745, 0,685, dan 0,630; sebagian besar coverage tambahan sudah diperoleh sebelum ekor ranking.

Dense indexer memberikan hasil terbaik pada semua metrik retrieval manual (Tabel V). Selisih recall dense terhadap sparse adalah 0,365, sedangkan terhadap hybrid adalah 0,045. Pada $K = 30$, recall dense untuk pertanyaan Inggris dan Indonesia masing-masing 0,810 dan 0,760; gap lintas bahasa 0,050 lebih kecil daripada hybrid (0,120) dan sparse (0,240). Hal ini konsisten dengan tujuan BGE-M3 untuk menemukan parafrasa multibahasa, tetapi tidak mengubah batasan bahwa kecocokan vektor bukan kesetaraan makna hukum.

Pada generation dengan $K = 10$, dense menghasilkan correctness/faithfulness 0,788/0,958, hybrid 0,711/0,946, dan sparse 0,508/0,881. Payload dense justru paling kecil, sekitar 3.399 token, dibandingkan hybrid 3.514 dan sparse 3.708. Jadi peningkatan correctness tidak dijelaskan oleh jumlah token

TABLE III
KONFIGURASI OFAT PADA DUA KORPUS

Korpus dan kelompok	Level yang dibandingkan	Parameter tetap
Manual-chunker	structure-aware hybrid; recursive; sliding-window	512 karakter, overlap 51 untuk recursive/sliding, hybrid indexer, parser dan embedding yang sama.
Manual-indexer	structure-aware dense; hybrid; structure-aware sparse	structure-aware chunker, BAAI/BGE-M3 untuk dense, lexical field dan corpus yang sama.
Hukum-chunker	legal-structure dense; recursive; sliding	parent hydration, 512 karakter untuk struktur legal, overlap 51 untuk recursive/sliding, dense indexer.
Hukum-indexer	legal-structure dense; structure-aware hybrid; structure-aware sparse	legal structure-aware chunker, corpus dan context reference yang sama; hybrid memakai RRF.

TABLE IV
HASIL KELOMPOK CHUNKER PADA USER MANUAL, $K = 30$

Strategi	R	P	N	M
Structure-aware hybrid	0,740	0,048	0,333	0,231
Recursive	0,625	0,032	0,291	0,216
Sliding-window	0,685	0,041	0,322	0,232

R: Recall, P: Precision, N: NDCG, M: MRR.

TABLE V
HASIL KELOMPOK INDEXER PADA USER MANUAL, $K = 30$

Strategi	R	P	N	M
Structure-aware dense	0,785	0,057	0,350	0,314
Hybrid	0,740	0,048	0,333	0,231
Structure-aware sparse	0,420	0,028	0,157	0,120

R: Recall, P: Precision, N: NDCG, M: MRR.

semata, melainkan oleh reference yang lebih lengkap dan lebih awal pada sepuluh evidence teratas.

B. Hasil pada Peraturan Pendidikan

Legal-structure dense mengalahkan sliding baseline pada recall sebesar 0,0509 (Tabel VI). Keunggulan tersebut berasal dari child chunk yang tetap membawa identitas Pasal dan konteks induk, bukan dari penambahan jumlah kandidat. Recursive berada dekat dengan dense karena kalimat yang bersebelahan sering tetap bersama; sliding hampir menyamai dense pada NDCG (0,5042 berbanding 0,5040), tetapi tidak menemukan sebanyak reference secara total.

Pada kelompok indexer, dense memperoleh recall 0,6854, sparse 0,6381, dan hybrid 0,6412. Hybrid memiliki MRR tertinggi, 0,5439, yang berarti satu reference pertama lebih sering muncul di posisi awal. Namun, tujuan utama pipeline adalah menyediakan sebanyak mungkin context reference untuk tahap berikutnya, sehingga dense tetap dipilih sebagai konfigurasi coverage. Perbedaan ini memperlihatkan bahwa recall dan posisi hit pertama mengukur sifat ranking yang berbeda.

Generation hukum memperkuat pola tersebut. Legal-structure dense menghasilkan correctness 0,690 dan faithfulness 0,972. Sparse menghasilkan 0,578/0,982, sedangkan hybrid menghasilkan 0,664/0,982. Faithfulness yang tinggi tidak berarti jawaban lengkap: evidence dapat mendukung klaim yang ditulis, tetapi tidak memuat seluruh rincian reference. Payload rata-rata ketiga indexer hampir sama, sehingga selisih

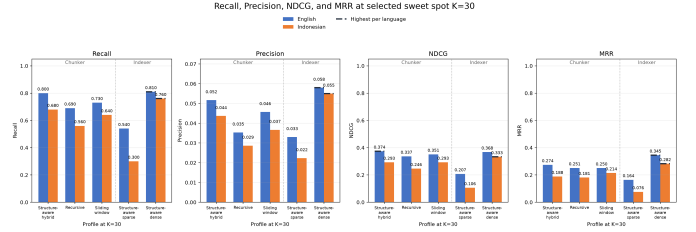


Fig. 5. Metrik seluruh konfigurasi hukum pada $K = 30$.

Knowledge-graph recall by configuration

Depth-two neighborhood coverage · seed documents excluded from the neighborhood

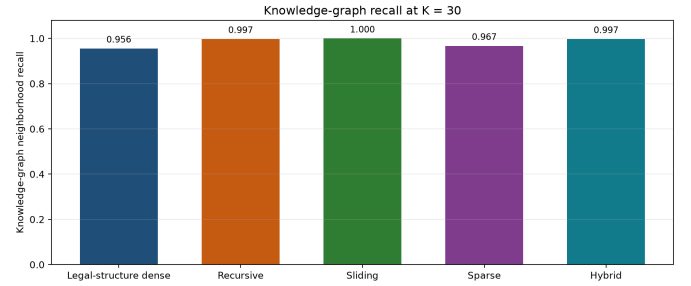


Fig. 6. Neighborhood recall legal graph pada $K = 30$.

correctness lebih tepat dibaca sebagai dampak coverage dan urutan evidence.

C. Perluasan dengan Legal Knowledge Graph

Eksperimen graf memetakan reference ke dokumen sumber, mengambil seed dari top-30 retrieval, lalu menelusuri neighborhood global sampai kedalaman dua. Pada benchmark hukum, graf memiliki 17 relasi berarah dan 24 relasi tak berarah. Neighborhood recall pada $K = 30$ adalah 0,9556 untuk dense, 0,9975 untuk recursive, 1,0000 untuk sliding, 0,9672 untuk sparse, dan 0,9975 untuk hybrid.

Nilai 1,0000 pada sliding tidak bertentangan dengan recall context reference yang lebih rendah. Satu dokumen seed dapat membuka beberapa dokumen tetangga melalui relasi perubahan, sehingga graf mengukur jangkauan lingkungan dan bukan ketepatan langsung setiap unit bukti. Temuan ini mendukung penggunaan graf sebagai proyeksi pelengkap: text retrieval memilih seed, sementara traversal menyediakan konteks lintas-dokumen yang eksplisit dan dapat ditelusuri.

TABLE VI
HASIL RETRIEVAL PADA CORPUS HUKUM, $K = 30$

Konfigurasi	Recall	Precision	NDCG	MRR
Legal-structure dense (kelompok chunker)	0,6854	0,1520	0,5040	0,5240
Recursive (kelompok chunker)	0,6730	0,1453	0,4909	0,4937
Sliding (kelompok chunker)	0,6345	0,1507	0,5042	0,4626
Structure-aware dense (kelompok indexer)	0,6854	0,1520	0,5040	0,5240
Structure-aware sparse	0,6381	0,1253	0,4262	0,4613
Structure-aware hybrid	0,6412	0,1307	0,4797	0,5439

D. Pembahasan dan Keterbatasan

Hasil dua korpus menunjukkan pola yang konsisten tetapi tidak identik. Structure-aware membantu ketika struktur sumber membawa informasi semantik, terutama pada legal-structure yang mempertahankan identitas Pasal dan parent context. Dense retrieval paling baik untuk coverage pada kedua domain, tetapi hybrid lebih baik untuk MRR hukum. Dengan demikian, tidak ada konfigurasi yang unggul pada seluruh metrik; tujuan tahap harus ditetapkan sebelum memilih komponen.

Kesimpulan ini bersifat kondisional pada corpus, pertanyaan, parser native-text, model BGE-M3, dan cutoff yang digunakan. Manual hanya mencakup sembilan produk Netgear dan 200 pertanyaan bilingual, sedangkan corpus hukum mencakup satu bidang dan 25 pertanyaan terverifikasi. OFAT tidak mengukur interaksi semua kombinasi parameter. Neighborhood recall juga tidak dapat dipakai sebagai pengganti context-reference recall. Perluasan corpus, anotasi relasi per pertanyaan, full factorial, dan pengujian model embedding hukum menjadi pekerjaan lanjutan yang wajar.

VI. KESIMPULAN

Paper ini menyajikan konstruksi basis pengetahuan modular untuk OMNI-RAG multi-domain, dengan Capstone hanya sebagai konteks integrasi. Kontrak artefak kanonik membuat keluaran parser, *chunker*, *indexer*, dan *knowledge graph* dapat dipertukarkan; *plugin* Registry dan *profile* memungkinkan strategi baru dipilih tanpa mengubah orkestrator. Dari empat strategi chunker, pemeliharaan struktur memberikan coverage lebih tinggi daripada fixed-window pada manual maupun peraturan. Dari tiga strategi indexer, dense BAAI/BGE-M3 memberikan Recall@30 tertinggi pada kedua corpus, sementara hybrid memiliki MRR hukum tertinggi. Legal graph deterministik memperluas seed retrieval ke neighborhood dokumen yang hampir atau sepenuhnya tercakup, tetapi metrik tersebut tetap dibaca terpisah dari recall context reference.

Secara empiris, konfigurasi utama adalah structure-aware dense untuk coverage manual (recall 0,785) dan legal-structure dense untuk coverage hukum (recall 0,6854). Hasil ini tidak menyatakan bahwa satu konfigurasi universal untuk semua domain; konfigurasi terbaik ditentukan oleh struktur dokumen dan tujuan metrik. Kontribusi utama penelitian adalah menunjukkan bagaimana keputusan tersebut dapat diuji secara terisolasi melalui kontrak, provenance, dan eksperimen OFAT.

Pengembangan berikutnya perlu memperluas corpus dan pertanyaan terverifikasi, menguji full factorial untuk interaksi

chunker-indexer, melatih atau mengevaluasi embedding yang lebih spesifik untuk bahasa hukum Indonesia, serta mengukur kualitas relasi graf per jenis dan per pertanyaan. Artefak run dan provenance yang sudah disimpan menyediakan dasar untuk eksperimen tersebut.

UCAPAN TERIMA KASIH

Penulis berterima kasih kepada pembimbing dan anggota tim proyek yang menyediakan konteks integrasi sistem. Paper ini hanya menyajikan kontribusi penulis pada konstruksi basis pengetahuan dan tidak mengklaim kinerja komponen sistem di luar ruang lingkup tersebut.

DAFTAR PUSTAKA

- [1] P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Apr. 12, 2021. arXiv: 2005.11401 [cs]. URL: <http://arxiv.org/abs/2005.11401> (visited on 12/24/2025). Pre-published.
- [2] Y. Huang and J. Huang. *A Survey on Retrieval-Augmented Text Generation for Large Language Models*. Aug. 23, 2024. arXiv: 2404.10981 [cs]. URL: <http://arxiv.org/abs/2404.10981> (visited on 12/24/2025). Pre-published.
- [3] P. Akarajardwong et al. *NitiBench: A Comprehensive Study of LLM Framework Capabilities for Thai Legal Question Answering*. Aug. 21, 2025. arXiv: 2502.10868 [cs]. URL: <http://arxiv.org/abs/2502.10868> (visited on 12/24/2025). Pre-published.
- [4] S. Robertson and H. Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *FNT in Information Retrieval 3.4* (2009), pp. 333–389.
- [5] V. Karpukhin et al. “Dense Passage Retrieval for Open-Domain Question Answering”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. arXiv: 2004.04906.
- [6] J. Chen et al. “M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, Aug. 2024, pp. 2318–2335.

- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. In: *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2009, pp. 758–759.
- [8] D. Edge et al. “From Local to Global: A Graph RAG Approach to Query-Focused Summarization”. In: *arXiv* (Apr. 24, 2024). arXiv: 2404.16130.
- [9] H. Han et al. *Retrieval-Augmented Generation with Graphs (GraphRAG)*. Jan. 8, 2025. arXiv: 2501.00309 [cs]. URL: <http://arxiv.org/abs/2501.00309> (visited on 12/22/2025). Pre-published.
- [10] Y. Gao, Y. Xiong, M. Wang, and H. Wang. “Modular RAG: Transforming RAG Systems into LEGO-like Reconfigurable Frameworks”. In: *arXiv* (July 26, 2024). arXiv: 2407.21059.
- [11] D. Garlan and M. Shaw. *An Introduction to Software Architecture*. Tech. rep. CMU-CS-94-166. Pittsburgh, PA: School of Computer Science, Carnegie Mellon University, 1994.